

Введение в проектирование информационных систем

Лекция 1 · 07.09.2026

Устное пояснение: **Организационное.** Это первая лекция дисциплины «Технологии проектирования информационно-вычислительных систем». Лектор — Тимофеев Александр Вадимович. Практику и лабораторные ведёт Кузенный Михаил Николаевич.

- Все вопросы к экзамену выдаются заранее; ФОС (фонд оценочных средств) — в наличии.
- Цель — получить оценку автоматически; для этого нужны посещаемость и своевременная сдача лабораторных и практических работ.
- Отметка присутствующих проводится один раз, переключкой по группам.

Устное пояснение: **Литература** (слайд 2):

1. Леоненков А. В. Самоучитель UML 2. — СПб.: БХВ-Петербург, 2007. — 576 с.
2. Гринченков Д. В., Дьяченко В. Б., Куций Д. Н. Введение в программную инженерию: учебное пособие / Южно-Российский государственный политехнический университет (НПИ) имени М. И. Платова. — Новочеркасск: ЮРГПУ (НПИ), 2016. — 91 с.
3. Иванов Д., Новиков Ф. Моделирование на UML [Электронный ресурс]. — СПб.: НИУ ИТМО, 2010. — 200 с.
4. Ларина Ю. А. Основы объектно ориентированного моделирования с использованием языка UML: учеб. пособие / Яросл. гос. ун-т им. П. Г. Демидова. — Ярославль: ЯрГУ, 2010. — 152 с.
5. Золотов С. Ю. Проектирование информационных систем. Учебное пособие. Часть 1. — Томск: Эль Контент, 2013.

Также допустимы любой учебник по UML, любой учебник по IDEF и презентации из интернета. Лектор особо рекомендует пособие Гринченкова с соавторами.

1 Информационная система и её жизненный цикл

Определение 1 (Информационная система) . Совокупность аппаратных, программных, информационных и организационных средств, которые совместно выполняют сбор, обработку, хранение и передачу информации.

Основные действия с информацией: сбор, преобразование, обработка, хранение, передача, представление. Как и у всякого объекта, у информационной системы есть свой *жизненный цикл*.

Устное пояснение: Основная идея информационно-вычислительной системы (ИВС) — осуществлять сбор, обработку, хранение и представление информации без искажения.

2 Состав и структура системы

Термины, которые используются в курсе: ИВС (информационно-вычислительная система), состав ИС, классификация.

Система разделяется на *основные элементы* и *связи* между ними. Количество элементов зависит от того, какую систему мы описываем. Например, в информационной системе университета есть база данных с людьми, база данных с их учётными записями, база данных с учебными (сессионными) данными и т. д.

- **Основные принципы** задают обязательные правила, по которым строится решение и оценивается его обоснованность.
- **Методика применения ИС** — последовательность действий от постановки задачи до проверки результата. Проверка результата обязательна: иначе не с чем сравнивать.
- **Критерий качества.** Качество оценивается не по внешнему виду, а по заранее определённым измеримым критериям. Пример: если от системы требуется представление данных в реляционном виде, в требованиях указывают вывод по 10 строк, по 50 строк и т. д.
- Любая ИС проектируется в соответствии с **техническим заданием**; при приёмке системы (в том числе как ВКР) сверяются с ТЗ.

3 Виды информационных систем

Информационные вычислительные системы (ИВС) делятся по назначению (слайд 3):

- **Учётные системы:** бухгалтерские (в т. ч. отдельно зарплата), складские, кадровые, материальных ресурсов, заказов и услуг.
- **Системы планирования и бюджетирования.**
- **Системы документооборота.**
- **Системы целевых, предметных бизнес-процессов.**
- **ИС обработки больших объёмов данных (Big Data):** web scraping, структурирование данных, в том числе машинное обучение.
- **Экспертные системы, базы знаний.**

Какие требуются навыки (skills) для проектирования ИВС?

- **Вербальное описание предметной области и постановка задачи** — разработка требований к ИВС.
- **Описание информационных потоков:** документы на входе/выходе, отчёты, внутренний обмен.
- **Проектирование интерфейсов и баз данных.**

Виды деятельности в организации:

Определение 2 (Операционная деятельность) . Циклические процессы, выполняемые одними и теми же сотрудниками и приносящие одни и те же результаты. Такие процессы подлежат автоматизации.

Определение 3 (Проектная деятельность) . Работа, имеющая начало и конец (время), заданные результаты (качество) и бюджет (стоимость) — безразмерных денег не бывает, бюджет учитывается всегда.

3.1 Четвёртая промышленная революция

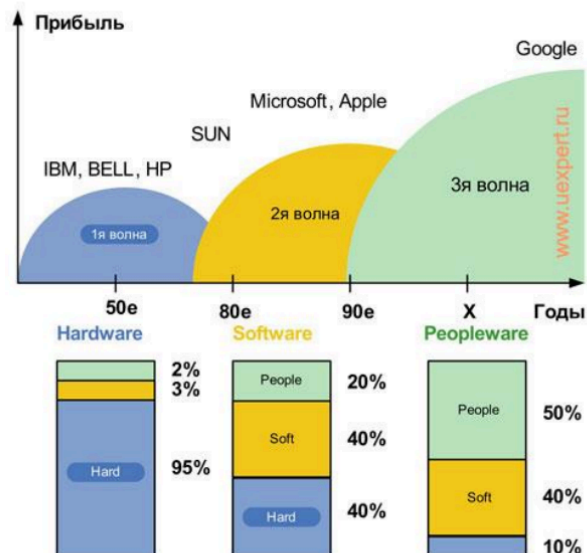
Четыре основные характеристики:

1. **Информатизация** — главенство информационной и электронной формы, доступ к информации «из каждого угла».
2. **Роботизация.**
3. **Большие данные.**
4. **Искусственный интеллект.**

Отсюда — активное развитие систем обработки больших данных, с разным уровнем доступа (непосредственный доступ, веб-доступ, обработка машинным обучением).

4 Тенденции развития индустрии программного обеспечения

Тенденции развития индустрии программного обеспечения



4

Слайд 1. Тенденции развития индустрии программного обеспечения. © www.uexpert.ru

По годам показана прибыль компаний: в 1950-х — «первая волна» (IBM, Bell, Hewlett-Packard), в 1980–90-х — «вторая волна» (Sun, Microsoft, Apple), затем — «третья волна» (Google). Вместе со сменой волн менялось распределение прибыли между аппаратными средствами (hardware), программным обеспечением (software) и людьми (peopleware): 95/3/2 % → 40/40/20 % → 10/40/50 %. Эволюция источника прибыли: сначала торговали «железом», затем — программным обеспечением, теперь основную прибыль приносят люди — чья компания привлекла больше специалистов, у той и прибыль выше.

5 Взгляд на создание успешных бизнес-продуктов

Определение 4 (Успешная компания) . Компания, которая вывела на рынок продукт, которым все хотят или вынуждены пользоваться.



5

Слайд 2. Взгляд на создание успешных продуктов. © Алан Купер, 2009

Для успеха продукта нужны три составляющие: *желанность* (что нужно людям), *осуществимость* (что мы способны создать) и *жизнеспособность* (чем продукт поддержит бизнес). Каждой стороне соответствует своя роль и модель:

- **Проектирование (проектировщик):** модель пользователя — мотив, поведение, отношение и сложности; проектирование продукта — проектный план, спецификация формы и поведения; итог — эффективная работа пользователей и принятие клиентами.
- **Менеджмент:** бизнес-модель — модель финансирования, прогнозы затрат и расходов; бизнес-план — маркетинговый план, план запуска (минимизируют, чтобы быстрее начать продавать) и план продаж (когда наступит окупаемость); итог — устойчивый бизнес.
- **Технологии:** техническая модель — базовые технологии, технологические компоненты (покупать или делать самим); технологический план — график разработки, технологическая спецификация; итог — сдача проекта. Разработку ведёт РМ, вывод продукта — Product Manager; перед сдачей продукта отдел QA (отдел качества) проверяет и тестирует.

Устное пояснение: На успех продукта влияют и внешние факторы: неграмотность населения, политика (обязательность использования — например, 1С).

Пример 1 (Novell, Apple, Microsoft) . Novell делала упор на технологию и уделяла мало внимания желанности продуктов, а потому оказалась уязвимой для конкурентов.

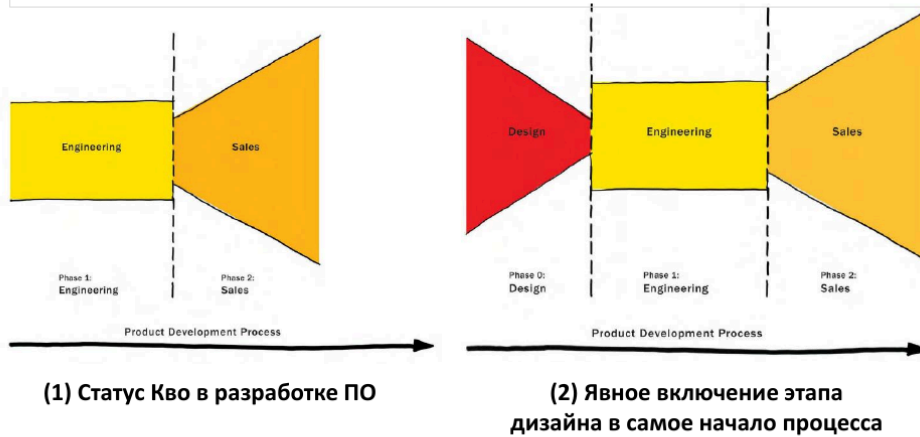
Apple делала ставку на желанность, но совершила множество грубых ошибок в бизнесе; тем не менее компанию поддерживает преданность пользователей, завоёванная вниманием к их опыту взаимодействия с продуктами Apple. Microsoft — из числа успешных в бизнесе компаний, однако она не смогла создать высококласные и желанные продукты; это открывает возможности для конкурентов.

6 Эволюция процесса разработки ПО

Проектирование целостного опыта взаимодействия, или эволюция процесса разработки ПО (1)

Проблема первых подходов:

- 1) Отсутствие представлений о пользователях
- 2) Конфликт между потребностями людей и приоритетами разработки
- 3) Отсутствие надёжного процесса проектирования желанных продуктов

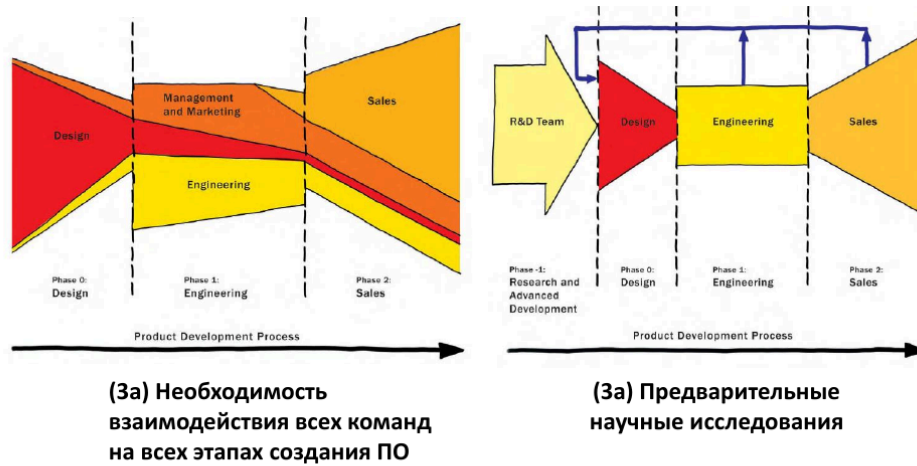


© Билл Бакстон, 2007

6

Слайд 3. Эволюция процесса разработки ПО (1). © Билл Бакстон, 2007

Проектирование целостного опыта взаимодействия, или эволюция процесса разработки ПО (2)



© Билл Бакстон, 2007

7

Слайд 4. Эволюция процесса разработки ПО (2). © Билл Бакстон, 2007

Проблема первых подходов к разработке (статус-кво: Engineering → Sales):

1. **Отсутствие представления о пользователях** — «надо сделать что-то, с чего все получают прибыль», без понимания, кому и зачем это нужно.
2. **Конфликт между потребностями людей и приоритетами разработки.** Люди хотят, чтобы сразу всё работало; приоритеты разработки определяются финансами. Решение — явное включение этапа дизайна в начало процесса (Design → Engineering → Sales) и цикл обратной связи: *сбор фидбэка → анализ отдельной группой → составление ТЗ → разработка → тестирование QA → новая версия → снова наблюдение*. Когда приложение становится популярным, от него начинают требовать новое.
3. **Отсутствие надёжного процесса проектирования желаемых продуктов** — никто не может гарантировать, что задуманное точно будет работать и приносить доход.

Дальнейшее развитие процесса требует взаимодействия всех команд (Design/Engineering/Management and Marketing/Sales) на всех этапах и предварительных научных исследований (этап R&D Team перед Design).

6.1 Инженеры и продавцы

- **Инженеры** отвечают за то, будет ли приложение вообще работать и жизнеспособно ли оно. Работающий, но невзрачный продукт — не всегда успешный, поэтому любой продукт должен иметь интерфейс, а у интерфейса — дизайн; на дизайн тоже нужно закладывать бюджет.
- **Продавцы** делятся на хороших и не очень; методы продаж — агрессивный и вербальный.

Устное пояснение: Устная иллюстрация (без слайда): басня И. А. Крылова «Лебедь, рак и щука». Когда дизайн лезет в продажи, менеджмент — в дизайн, инженеры — в дизайн

и т. д., вывод продукта затягивается, а любая задержка — это прямые финансовые потери (разработчиков и дизайнеров надо оплачивать ежемесячно).

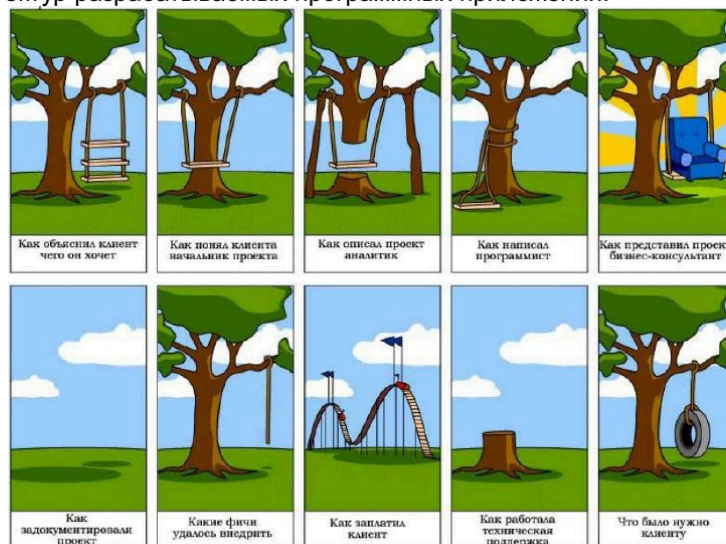
6.2 Что ещё нужно для успеха

- **Исследование рынка до начала разработки** — требует специалистов и денег. В работах опираются на сторонние данные либо на собственную фокус-группу (метрики, например Яндекс.Метрика).
- **Умение работать в команде** — главный навык для программиста и, тем более, для проектировщика ИВС: «без коллектива можно построить только скворечник, в лучшем случае — собачью будку» (слайд 8). Нужен организованный процесс: люди, которые собирают команду на планёрках и гасят конфликты; отдельная роль — психолог (у ИТ-специалистов бывает профессиональное выгорание).

Проблемы программной инженерии

Причины возрастания рисков:

- **низкая квалификация** персонала рабочих групп и менеджеров проектов;
- - одно из главных мест занимает **отсутствие адекватных моделей** и архитектур разрабатываемых программных приложений.



13

Слайд 5. Как объяснил клиент чего он хочет / как понял клиента начальник проекта / как описал проект аналитик / как написал программист / как представил проект бизнес-консультант / как задокументировали проект / какие фичи удалось внедрить / как заплатил клиент / как работала техническая поддержка / что было нужно клиенту

Несогласованность команды влияет прежде всего на качество и на репутацию компании: контора, которая так себя ведёт, обычно уходит с рынка (кроме государственных).

7 Программная инженерия

Определение 5 (Программная инженерия) . Систематический, строго определённый и измеримый подход к разработке, эксплуатации и поддержке программного обеспе-

чения (определение дано IEEE Standard Glossary of Software Engineering Terminology; IEEE — Institute of Electrical and Electronics Engineers).

Устное пояснение: Термин **software** (программное обеспечение, ПО) ввёл в 1958 году всемирно известный математик и статистик Джон Тьюки (John Tukey). Термин **software engineering** впервые был озвучен в октябре 1968 года на конференции подкомитета НАТО по науке и технике (г. Гармиш, Германия).

7.1 SWEBOOK

В 2004 году в отрасли был создан основополагающий труд «Руководство к своду знаний по программной инженерии» (The Guide to the Software Engineering Body of Knowledge, сокращённо — SWEBOOK), в котором собраны основные теоретические и практические знания, накопленные в этой отрасли.

SWEBOOK аккумулирует в себе:

- представление сообщества о том, что должен знать и какими навыками владеть инженер-программист;
- базу с целью формирования экзаменационных программ для лицензирования специалистов;
- критерии аккредитации этих программ и учебных планов непрерывного образования.

Цель — задать границы программной инженерии как сферы профессиональной деятельности и снять *априорную неопределённость* (когда неизвестно, «за что хвататься», и специалисты не могут договориться о терминологии).

Устное пояснение: Практика подготовки инженеров-программистов в США: знания и навыки по SWEBOOK, которыми должны обладать разработчики, имеющие степень бакалавра и четыре года практической работы, — для успешного прохождения лицензионного экзамена.

7.2 Десять областей знаний SWEBOOK

1. Требования к программному обеспечению (Software Requirements). Инструкции должны быть удобочитаемыми; английский язык — один из обязательных вариантов.
2. Проектирование программного обеспечения (Software Design).
3. Конструирование программного обеспечения (Software Construction) — «повар пробует блюдо, прежде чем отдать».
4. Тестирование программного обеспечения (Software Testing).
5. Сопровождение программного обеспечения (Software Maintenance). На сопровождении зарабатывается больше всего денег: разработчик делает ТЗ один раз, а доработки («лишняя кнопка», изменение поведения) оплачиваются постоянно.
6. Управление конфигурацией программного обеспечения (Software Configuration Management).
7. Управление в программной инженерии (Software Engineering Management) — менеджмент для управления людьми.
8. Процесс программной инженерии (Software Engineering Process) — весь процесс от начала до конца; без него участники «едут в автобусе, не зная, куда».

9. Инструменты и методы программной инженерии (Software Engineering Tools and Methods) — как именно реализовать процесс.
10. Качество программного обеспечения (Software Quality) — отдел качества сверяет результаты тестирования с техническим заданием и возвращает работу на доработку.

7.3 Проблемы программной инженерии

Одновременно с увеличением стоимости информационных проектов заметно возросли риски успешного их выполнения: большие проекты требуют большого числа людей. В настоящее время часто говорят о *кризисе программирования*, который принимает хронические формы — нет общего знаменателя, каждый работает так, как ему проще (современная тенденция — вайб-кодинг).

США тратит ежегодно более \$300 млрд на более чем 200 тыс. проектов разработки ПО в сфере ИТ. Потери от недополученного эффекта внедрения ПО измеряются триллионами.

Проекты делят на три категории:

Успешные вовремя и в рамках бюджета выполнен весь намеченный фронт работ.

Проблемные нарушение сроков, перерасход бюджета и/или выполняется не всё, что требовалось.

Проваленные не были доведены до конца из-за перерасхода средств, бюджета, качества.

Причины возрастания рисков:

- низкая квалификация персонала рабочих групп и менеджеров проектов;
- отсутствие адекватных моделей и архитектур разрабатываемых программных приложений — система «не спроектирована», всё держится на словах, а «слова к делу не пришьёшь».

8 Необходимость проектирования (моделирования) ИС

Все эти особенности приводят к настоящей необходимости моделирования структуры и процесса функционирования программных систем *до начала написания кода*. Непременным условием успешного завершения проекта становится построение предварительной визуальной модели программной системы (аналогия — план эвакуации: по плану человек действует, а на словесные команды в стрессовой ситуации реагирует плохо).

Определение 6 (Цель проектирования (моделирования) ИВС) . Представить проект в формализованном ясном виде, одинаково понятном как заказчикам, так и разработчикам, + дать язык общения группе специалистов.

9 Основные понятия моделирования систем и программных приложений

Определение 7 (Модель) . Абстрактное представление системы или объекта на некотором языке или в графической форме.

Определение 8 (Структура системы) . Устойчивая во времени совокупность взаимосвязей между её элементами или компонентами. Структура связывает воедино все элементы и препятствует распаду системы на отдельные составляющие: по отдельности элементы не работают, работает только всё вместе.

- Структура системы может отражать вложенность элементов одной системы в другую. Вложенная система называется **подсистемой**, более крупная — **метасистемой**.
- Структура программных систем на верхнем уровне называется **архитектурой**, которая содержит наиболее общие структурные компоненты приложений и взаимосвязи между ними.

Определение 9 (Процесс разработки моделей) . Поуровневый спуск от наиболее общих моделей концептуального уровня к более частным и детальным представлениям логического и физического уровня. На каждом этапе процесса разработки модели последовательно дополняются всё большим количеством деталей.

Модели бывают разные — материальные и нематериальные, искусственные и естественные, декоративные и математические.

Пример 2 . Пластмассовые игрушечные автомобильчики — это не что иное, как материальная искусственная *декоративная модель* реального автомобиля: она даёт ребёнку представление об автомобиле, поскольку отображает его характерные черты (наличие четырёх колёс, кузова, дверей, окон, способность ехать). В ходе медицинских исследований опыты на животных часто предшествуют клиническим испытаниям медицинских препаратов на людях: в таком случае животное выступает в роли *материальной естественной модели* человека. Уравнение

$$mg - a \frac{dx}{dt} = m \frac{d^2x}{dt^2}$$

— *математическая модель*, описывающая движение материальной точки под действием силы тяжести.

10 Основные понятия курса «Технологии проектирования ИВС»

Методология (methodology)	Совокупность принципов и методик разработки программных систем и приложений, определяющих процесс построения моделей при выполнении соответствующих проектов.
Модель системы (ИВС)	Графическое описание (рисунок) состава и функционала системы в виде диаграмм (аналог блок-схемы) с текстовым пояснением.
Цель разработки модели (purpose)	Для чего разрабатывается модель (описание существующих бизнес-процессов и/или новых, оптимизация обработки информации, формализация обработки информации, реинжиниринг). <i>Не путать с целью разработки системы.</i>

Точка зрения модели (viewpoint)	Для кого разрабатывается модель: какому должностному лицу она будет дана на рассмотрение.
Область моделирования (scope)	Какие функции и компоненты системы расписаны детально, а какие будут считаться внешними воздействиями.
Проектирование и/или моделирование систем (ИВС)	Текстовое описание цели, точки зрения, области моделирования + разработка модели системы с графической частью и текстовым пояснением.

Диаграмма	Графическое изображение функционального блока (подсистемы).
Графическая нотация	Отображение в виде рисунка.
Графические примитивы	Элементы диаграммы (стрелки, прямоугольники, овалы, специальные графические символы).
Методология структурного анализа и проектирования/моделирования системы (ИВС)	Набор понятий, терминов, графических примитивов и правил построения модели системы, её графического отображения и анализа.
Стандарт проектирования системы	Формализация некоторой методологии в единых терминах и правилах (BPMN, IDEF0, IDEF3, IDEF1X, UML 2.0, ARIS). IDEF — Integrated computer aided manufacturing DEFinition.
CASE-технологии проектирования систем	Computer Aided System Engineering — программные средства для автоматизированного проектирования и анализа систем.
AS-IS, SHOULD-BE, TO-BE процессы	Существующие (AS-IS), идеализированные (SHOULD-BE) и вновь создаваемые (TO-BE) процессы и функции системы.

На практике курса используются в первую очередь нотации IDEF0 и IDEF3, возможно — UML.

Устное пояснение: Лекция прервана по регламенту времени на середине этого раздела (слайд 18 из 24). Оставшаяся часть презентации — план проектирования программного продукта, фазы проектирования ИВС (инициация, определение, выполнение, завершение) и бизнес-план проекта — по словам лектора, относится к следующей лекции и в этот конспект не включена.